Kernel density plot

I **kernel density plot** forniscono la rappresentazione grafica della densità di probabilità di una distribuzione o più precisamente della stima kernel di detta densità di probabilità e possono essere utilizzati in alternativa o accanto ai tradizionali istogrammi dei quali rappresentano una importante evoluzione.

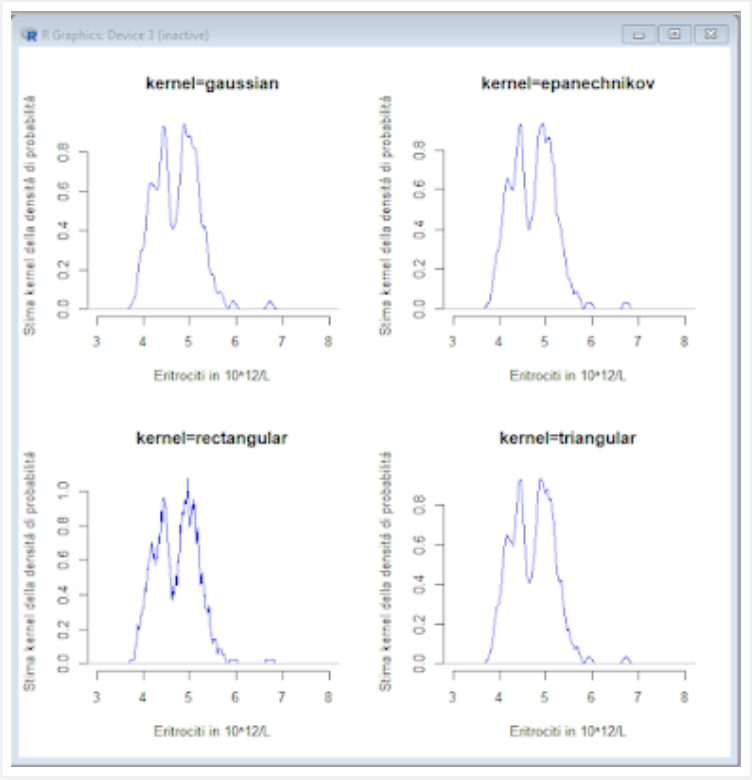
Il metodo non parametrico [1] impiegato per realizzare i kernel density plot – invece di suddividere i dati in classi e rappresentarli sotto forma di barre come negli istogrammi – prevede di collocare in corrispondenza di ciascun dato una piccola *gobba* (**bump**), determinata da un *fattore di forma* (**kernel**), e di sommare tutte le gobbe impiegando un fattore di smussamento detto *ampiezza di banda* (**bandwidth**). La curva risultante, il kernek densiy plot, riporta quindi sulle ascisse `x` i valori della variabile oggetto di studio espressi in una *scala numerica continua* [2] e sulle ordinate `y` la frequenza di tali valori espressa come *[stima kernel della] densità di probabilità della distribuzione*.

Da notare che gli esempi riportati sono stati realizzati senza la necessità di installare pacchetti aggiuntivi ma impiegando solamente le *funzioni statistiche e grafiche di base* di **R**.

Iniziamo esaminando il primo fattore che condiziona l'aspetto di un kernel density plot, il **fattore di forma** (*kernel*) impiegando la concentrazione degli eritrociti (globuli rossi) nel sangue rilevata in 202 atleti australiani riportata nella colonna/variabile **rcc** della tabella **ais** inclusa nel pacchetto **DAAG**. Accertatevi di avere installato il pacchetto o in alternativa procedete come indicato in [3].

Copiate e incollate questo script nella `Console di R` e premete `↵ Invio`:

```
# KERNEL DENSITY PLOT effetto del fattore di forma (kernel)
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
par(mfrow=c(2,2)) # suddivide la finestra in quattro quadranti,  uno per grafico
#
kdpg <-density(ais$rcc, kernel=c("gaussian"), bw=0.05, adjust=1) # calcola il kernel density plot
plot(kdpg, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="kernel=gaussian", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
kdpe <-density(ais$rcc, kernel=c("epanechnikov"), bw=0.05, adjust=1) # calcola il kernel density plot
plot(kdpe, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="kernel=epanechnikov", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
kdpr <-density(ais$rcc, kernel=c("rectangular"), bw=0.05, adjust=1) # calcola il kernel density plot
plot(kdpr, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="kernel=rectangular", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
kdpt <-density(ais$rcc, kernel=c("triangular"), bw=0.05, adjust=1) # calcola il kernel density plot
plot(kdpt, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="kernel=triangular", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
```



L'utilizzo del fattore di forma (*kernel*) è illustrato con quattro grafici, ciascuno generato impiegando:
1) la funzione **density()** con questi argomenti:
→ **ais$rcc** per indicare la variabile da analizzare;
→ **kernel="..."** per indicare il fattore di forma da applicare;
→ **bw=...** per indicare l'ampiezza di banda da applicare;
→ **adjust=...** per indicare il fattore di moltiplicazione da applicare a **bw**, cosa che ne facilita la personalizzazione;
2) la funzione **plot()** che ha come argomenti:
→ l'oggetto generato alla riga precedente dalla funzione **density()** e contenente i dati necessari per tracciare il grafico ;
→ **frame.plot=FALSE** per togliere la cornice rettangolare che delimita il grafico;
→ **xlim=c(... , ...)** per indicare valore minimo e massimo da applicare all'asse delle ascisse;
→ **col="..."** per applicare un colore specifico alla traccia del grafico (valore di default = **"black**");
→ **lwd=...** per indicare lo spessore della linea impiegata per rappresentare il grafico (valore di default = **1**);
→ **main="..."**, **xlab="..."** e **ylab="..."** per specificare titolo, etichetta dell'asse delle `x` ed etichetta dell'asse delle `y`.

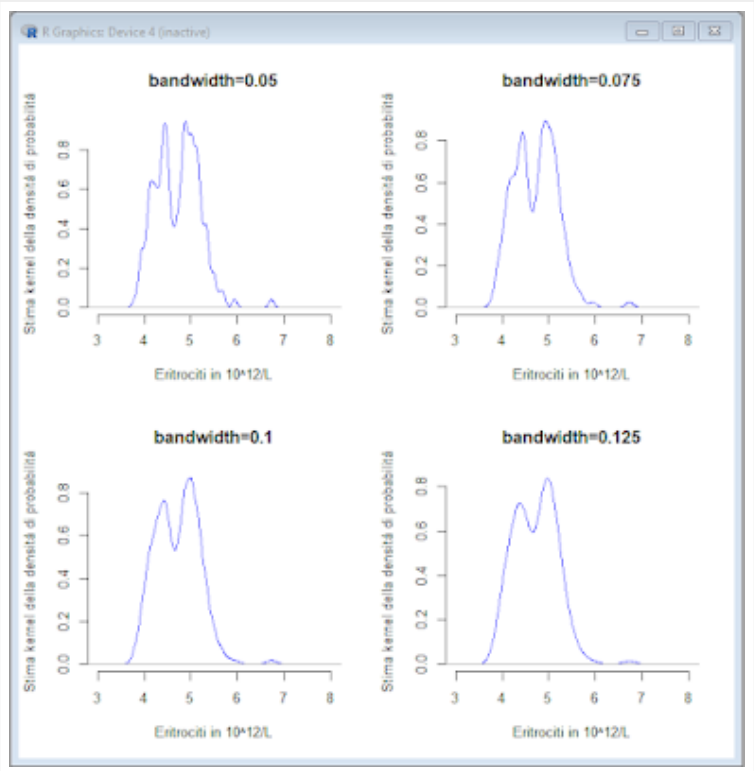
Nella funzione **density()** il fattore di forma viene specificato con l'argomento **kernel="..."** che può assumere i seguenti valori: **"gaussian"**, **"epanechnikov"**, **"rectangular"**, **"triangular"**, **"biweight"**, **"cosine"**, **"optcosine"** [4].

Questo significa che l'algoritmo che genera il kernel density plot sostituisce ciascun valore della variabile in esame con una *gobba* che nei nostri quattro casi ha la forma di una curva gaussiana (**"gaussian"**), di un vertice di parabola con convessità diretta verso l'alto (**"epanechnikov"**), di un rettangolo (**"rectangular"**), di un triangolo (**"triangular"**), forme che risultano evidenti nei due valori isolati e singoli visibili sulla destra in ciascun grafico.

Nota bene: lasciando fissa l'ampiezza di banda **bw=0.05** e variando il fattore di forma si vede che la somma di 202 rettangoli porta a un risultato più irregolare e frastagliato della somma di altrettante curve gaussiane, vertici di parabola o triangoli, ma in ogni caso il kernel density plot risultante mostra una distribuzione *bimodale* dei dati [5].

Vediamo ora il secondo fattore che condiziona l'aspetto di un kernel density plot, e cioè l'**ampiezza di banda** (*bandwidth*). Copiate e incollate questo script nella `Console di R` e premete `↵ Invio`:

```
# KERNEL DENSITY PLOT effetto dell'ampiezza di banda (bandwidth)
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
par(mfrow=c(2,2)) # suddivide la finestra in quattro quadranti,  uno per grafico
#
kdp50 <-density(ais$rcc, kernel=c("gaussian"), bw=0.05, adjust=1) # calcola il kernel density plot
plot(kdp50, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="bandwidth=0.05", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
kdp75 <-density(ais$rcc, kernel=c("gaussian"), bw=0.075, adjust=1) # calcola il kernel density plot
plot(kdp75, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="bandwidth=0.075", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
kdp100 <-density(ais$rcc, kernel=c("gaussian"), bw=0.1, adjust=1) # calcola il kernel density plot
plot(kdp100, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="bandwidth=0.1", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
kdp125 <-density(ais$rcc, kernel=c("gaussian"), bw=0.125, adjust=1) # calcola il kernel density plot
plot(kdp125, frame.plot=FALSE, xlim=c(3,8), col="blue", lwd=1, main="bandwidth=0.125", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
#
```



Nello script precedente abbiamo variato il fattore di forma lasciando fissa l'ampiezza di banda **bw=0.05**. Qui invece mantenendo fisso il fattore di forma **kernel="gaussian"** abbiamo fatto variare l'ampiezza di banda, portandola dal valore iniziale **bw=0.05** prima a **bw=0.075** poi a **bw=0.1** e infine a **bw=0.125**.

Risulta così documentato l'effetto dell'ampiezza di banda sul grafico: aumentandola aumenta lo smussamento del kernel density plot risultante, mentre diminuendola il grafico si adatta viepiù ai singoli punti e diventa perciò irregolare.

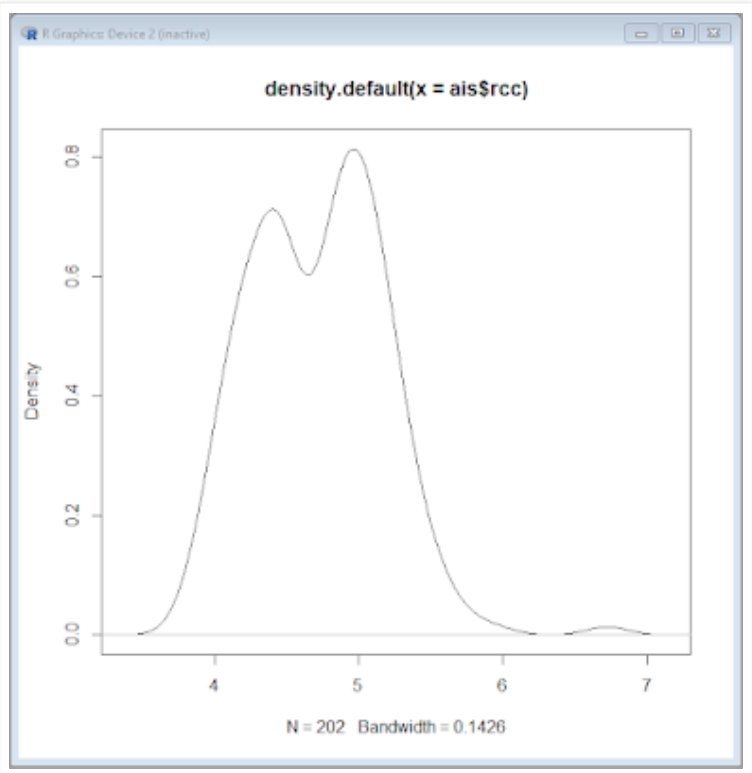
Nota bene: nel nostro caso il valore **bw=0.05** porta ad un eccessivo adattamento della curva ai singoli punti (*overfitting*) rendendola irregolare mentre il valore **bw=0.125** inizia a mostrare una minor separazione tra le due sottoclassi e quindi ci sta spostando verso un adattamento della curva ai dati troppo ridotto (*underfitting*) a causa dell'eccessivo fattore di smussamento. A questo punto la scelta tra un **bw=0.075** che conserva un maggior dettaglio e un **bw=0.1** che comunque mantiene l'informazione rilevante diventa una scelta personale, che dipende anche da quello che si intende comunicare e discutere in merito ai dati.

Dopo averne brevemente illustrato razionale e tecniche soggiacenti, possiamo passare a qualche esempio pratico di applicazione e personalizzazione dei kernel density plot realizzati impiegando solamente le *funzioni statistiche e grafiche di base* di **R**, aggiungendo al termine un esempio che impiega le funzioni contenute nel pacchetto aggiuntivo **sm** che consentono di realizzare facilmente kernel density plot multipli sovrapposti.

Gli esempi sono stati riportati in script separati, in modo che quelli che più interessano possano essere salvati indipendentemente l'uno dall'altro ed essere successivamente ripresi per analizzare i propri dati semplicemente eliminando la prima riga di codice e sostituendo **ais$rcc** con il nome della propria variabile.

Copiate e incollate questo primo script nella `Console di R` e premete `↵ Invio`:

```
# KERNEL DENSITY PLOT SEMPLICE
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
plot(density(ais$rcc)) # traccia il kernel density plot
#
```



Il kernel density plot più semplice che è possibile realizzare con **R** prevede, dopo avere caricato i dati e aperto e inizializzato una nuova finestra grafica, una sola riga di codice nella quale:

→ la funzione **density()**, che ha come unico argomento il nome della variabile da analizzare **ais$rcc**, genera la stima kernel della densità di probabilità necessaria per tracciare il grafico;
→ la funzione **plot()** impiega i dati generati dalla funzione **density()** per tracciare il grafico.

Se digitate

str(density(ais$rcc))

potete visualizzare la struttura dei dati generati dalla funzione **density()**

```
> str(density(ais$rcc))
List of 7
 $ x        : num [1:512] 3.37 3.38 3.39 3.39 3.4 ...
 $ y        : num [1:512] 0.000214 0.000251 0.000297 0.000349 0.000409 ...
 $ bw       : num 0.143
 $ n        : int 202
 $ call     : language density.default(x = ais$rcc)
 $ data.name: chr "ais$rcc"
 $ has.na   : logi FALSE
 - attr(*, "class")= chr "density"
```

mentre se digitate

density(ais$rcc)$x

e

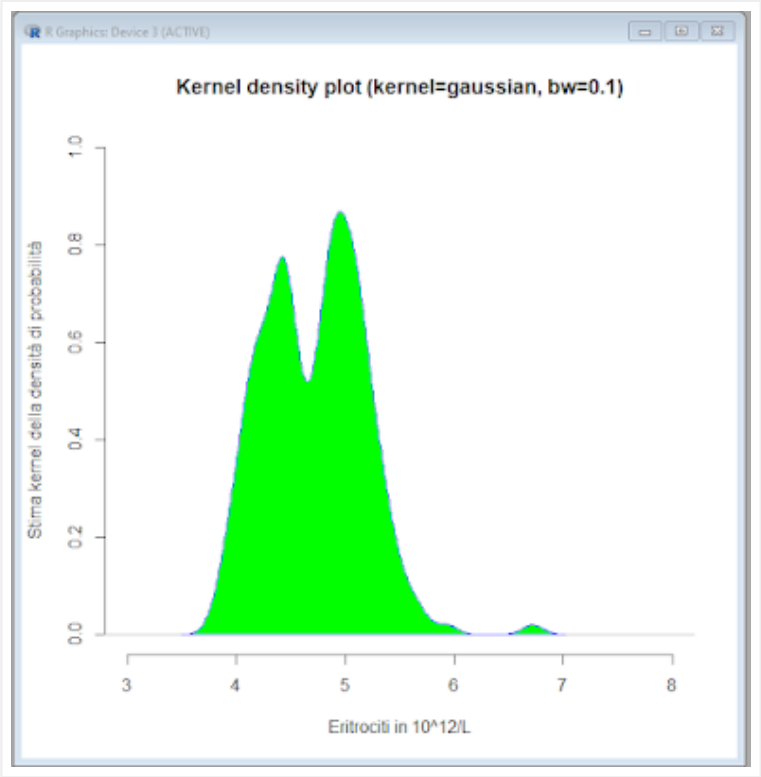
density(ais$rcc)$y

potete visualizzare rispettivamente in dettaglio tutti i 512 valori delle `x` e gli altrettanti valori delle `y` generati dalla funzione **density()** che la funzione **plot()** impiega per tracciare il grafico.

Nota bene: lo script contiene una soluzione minima che può essere utile quando si voglia effettuare una rapida analisi preliminare dei dati, ma non solo, perché la rappresentazione ottenuta con gli argomenti di default previsti da **R** fornisce, oltre al numero `N` dei dati, un'informazione importante: l'ampiezza di banda (*bandwidth*) impiegata per tracciare il grafico, che la funzione **density()** calcola automaticamente di volta in volta.

Per realizzare un kernel density plot personalizzato ora copiate e incollate questo script nella `Console di R` e premete `↵ Invio`:

```
# KERNEL DENSITY PLOT PERSONALIZZATO
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
kdp <-density(ais$rcc, kernel=c("gaussian"), bw=0.1, adjust=1) # calcola il kernel density plot
plot(kdp, frame.plot=FALSE, xlim=c(3,8), ylim=c(0,1), main="Kernel density plot (kernel=gaussian, bw=0.1)", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot
polygon(kdp, col="green", border="blue", lwd=1) # colora il kernel density plot
#
```



Qui la funzione **density()** è stata arricchita con gli argomenti:
→ **ais$rcc** per indicare la variabile da analizzare;
→ **kernel="gaussian"** per indicare che il fattore di forma da applicare è la curva gaussiana;
→ **bw=0.1** per indicare l'ampiezza di banda da applicare;
→ **adjust=1** per indicare il fattore di moltiplicazione da applicare a **bw**, cosa che in questo caso lascia **bw** immodificato.

La funzione **plot()** è stata personalizzata con gli argomenti già commentati nel primo script.

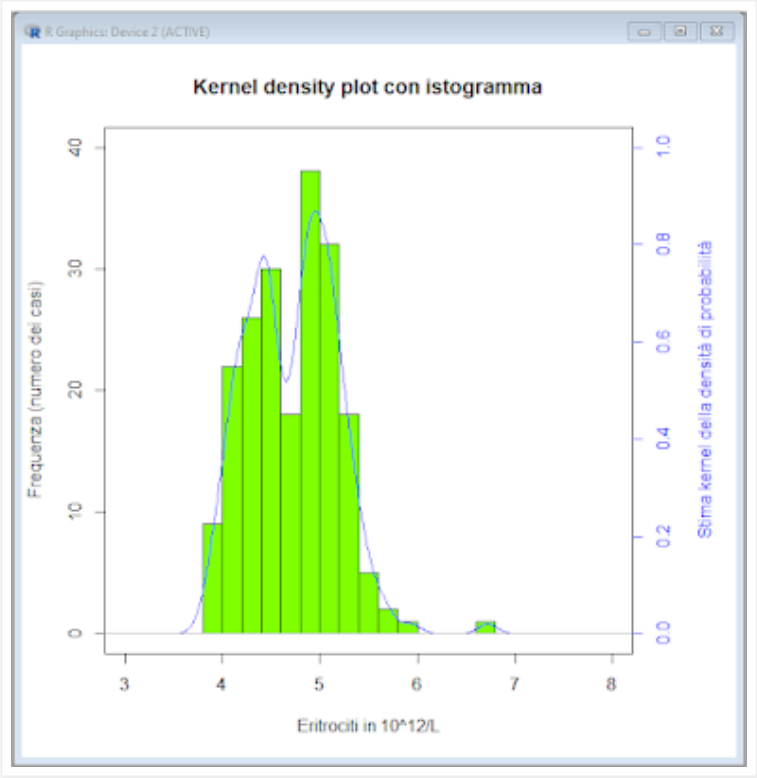
La novità è la funzione **polygon()** che consente di colorare il grafico specificando:
→ **kdp** per la variabile da analizzare;
→ **col="green"** per il colore di riempimento del kernel density plot;
→ **border="blue"** per il colore della linea che delimita il kernel density plot;
→ **lwd=1** per lo spessore della linea che delimita il kernel density plot.

Digitate **help(polygon)** nella `Console di R` se volete approfondire la conoscenza di questa funzione.

A questo punto vediamo come fare una cosa che prima o poi viene in mente a tutti: sovrapporre un kernel density plot e l'istogramma corrispondente. Copiate e incollate questo script nella `Console di R` e premete `↵ Invio`:

```
# KERNEL DENSITY PLOT CON L'ISTOGRAMMA CORRISPONDENTE e ordinate separate
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
par(mar=c(5.1,4.1,4.1,5.1)) # aumenta il margine destro da 2.1 a 5.1
#
hist(ais$rcc, xlim=c(3,8), ylim=c(0,40), freq=TRUE, col="chartreuse", breaks="FD",
main="Kernel density plot con istogramma", xlab="Eritrociti in 10^12/L",
ylab="Frequenza (numero dei casi)") # traccia l'istogramma
par(new=TRUE, ann=FALSE) # consente la sovrapposizione del grafico successivo
#
kdp <-density(ais$rcc, kernel=c("gaussian"), bw=0.1, adjust=1) # calcola il kernel density
plot
```

plot(kdp, yaxt="n", xlim=c(3,8), ylim=c(0,1), col="blue", lwd=1) # traccia il kernel density plot
axis(4, col.ticks="blue", col.axis="blue") # riporta l'asse delle y del kernel density plot sulla destra
mtext("Stima kernel della densità di probabilità", side=4, line=3, cex=1, col="blue") # aggiunge la legenda all'asse delle y sulla destra
#
par(mar=c(5.1,4.1,4.1,2.1)) # ripristina i margini di default
#



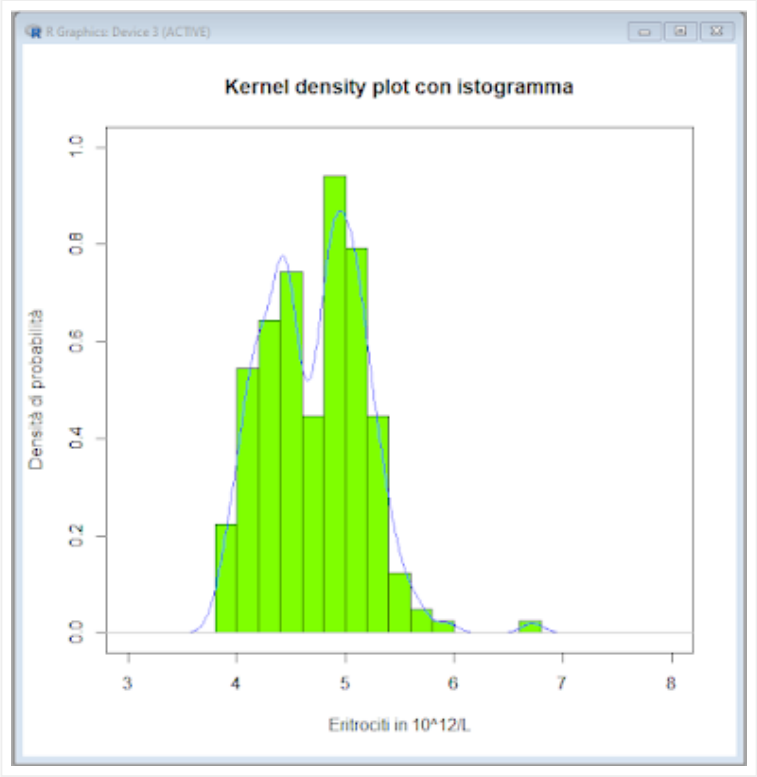
Vogliamo sovrapporre istogramma e kernel density plot riportando <u>due assi delle ordinate separati e con scale diverse</u>:
→ l'asse delle `y` sulla sinistra riferito all'istogramma e che riporta la *frequenza* espressa come numero dei casi;
→ l'asse delle `y` sulla destra riferito al kernel density plot e che riporta la *stima kernel della densità di probabilità*.

Ecco le principali osservazioni sugli argomenti e sulle funzioni impiegate:
→ come prima cosa con la funzione **par()** il margine sulla destra del grafico (il quarto valore dell'argomento **mar**) viene aumentato dal valore di default **2.1** a **5.1** per creare lo spazio necessario per la legenda dell'asse delle `y` del kernel density plot che verrà posizionato appunto sulla destra;
→ nella funzione **hist()** viene riportato in ordinate il numero dei casi ponendo **freq=TRUE**;
→ il numero delle classi **breaks** viene calcolato con la regola di Freedman-Diaconis (**FD**);
→ la funzione **par(new=TRUE, ann=FALSE)** specifica che il grafico successivo verrà sovrapposto (**new=TRUE**) ma che <u>non</u> devono essere annotati automaticamente titolo e legende (**ann=FALSE**), perchè questo lo faremo con le righe successive;
→ con la funzione **plot()** il kernel density plot viene sovrapposto all'istogramma specificando con **yaxt="n"** che <u>non</u> deve essere riportato l'asse delle `y` che si sovrapporrebbe a quello dell'istogramma e che invece noi aggiungeremo ora sulla destra;
→ con la funzione **axis()** riportiamo l'asse delle `y` del kernel density plot nella posizione **4** cioè sulla destra (nella grafica di base di **R** i margini sono 1=margine inferiore, 2=margine sinistro, 3=margine superiore, 4=margine destro) e riportiamo in colore **"blue"** sia le tacche sia i valori della scala;
→ con la funzione **mtext()** aggiungiamo sulla destra (**side=4**) la legenda dell'asse delle `y` del kernel density plot, posizioniamo l'etichetta **"Stima kernel della densità di probabilità"** sulla **line=3** (la linea 0 contiene le tacche, la 1 contiene i valori numerici, la 2 è volutamente lasciata vuota), riportiamo i caratteri nella dimensione standard (**cex=1**) e in colore blu (**col="blue"**);

→ con la funzione **par()** ripristiniamo i margini di default, di fatto riportiamo quello di destra (il quarto valore dell'argomento **mar**) da **5.1** a **2.1**.

Se preferite impiegare un'unica scala sia per l'istogramma sia per il kernel density plot, esprimendo per entrambi le ordinate in termini di *densità di probabilità*, copiate e incollate nella `Console di R` questo script e premete ↵ `Invio`:

```
# KERNEL DENSITY PLOT CON L'ISTOGRAMMA CORRISPONDENTE e ordinate comuni
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
hist(ais$rcc, xlim=c(3,8), ylim=c(0,1), freq=FALSE, breaks="FD", col="chartreuse",
main="Kernel density plot con istogramma", xlab="Eritrociti in 10^12/L", ylab="Densità
di probabilità") # traccia l'istogramma
par(new=TRUE, ann=FALSE) # consente la sovrapposizione del grafico successivo
#
kdp <-density(ais$rcc, kernel=c("gaussian"), bw=0.1, adjust=1) # calcola il kernel density
plot
plot(kdp, xlim=c(3,8), ylim=c(0,1), col="blue", lwd=1) # sovrappone all'istogramma il kernel
density plot
#
```

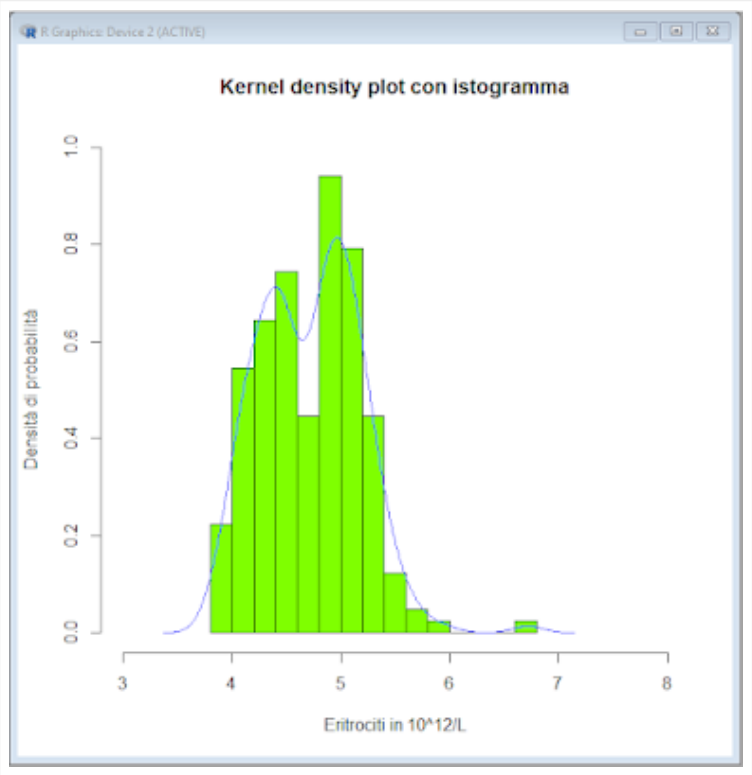


Come si vede sono ora sufficienti (tolte le prime due) quattro righe di codice:
→ la prima per tracciare con la funzione **hist()** l'istogramma, per gli argomenti del quale si rimanda al bisogno al post Istogrammi;
→ la seconda per consentire con la funzione **par()** la sovrapposizione all'istogramma di un nuovo grafico;
→ la terza per calcolare con la funzione **density()** il kernel density plot;
→ la quarta per tracciare con la funzione **plot()** il kernel density plot calcolato con la funzione **density()**.

Ma potete anche semplificare ulteriormente il codice impiegando questa sintetica variante:

```
# KERNEL DENSITY PLOT CON L'ISTOGRAMMA CORRISPONDENTE e ordinate comuni
#
```

library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
hist(ais$rcc, xlim=c(3,8), ylim=c(0,1), freq=FALSE, breaks="FD", col="chartreuse", main="Kernel density plot con istogramma", xlab="Eritrociti in 10^12/L", ylab="Densità di probabilità") # traccia l'istogramma
#
lines(density(ais$rcc), xlim=c(3,8), ylim=c(0,1), col="blue", lwd=1) # sovrappone all'istogramma il kernel density plot
#



Come si vede sono ora sufficienti:
→ la funzione **hist()** per tracciare l'istogramma;
→ la funzione **lines()** per sovrapporre all'istogramma il kernel density plot calcolato con **density(ais$rcc)**.

In questo caso il grafico è un po' più spartano e il kernel density plot risulta lievemente diverso dai due precedenti in quanto per semplificare al massimo il codice non viene imposto l'argomento **bw=0.1** ma viene lasciato che sia **R** a calcolarne il valore. Se digitate

density(ais$rcc)$bw

vi viene mostrato il valore di banda passante calcolato automaticamente da **R**

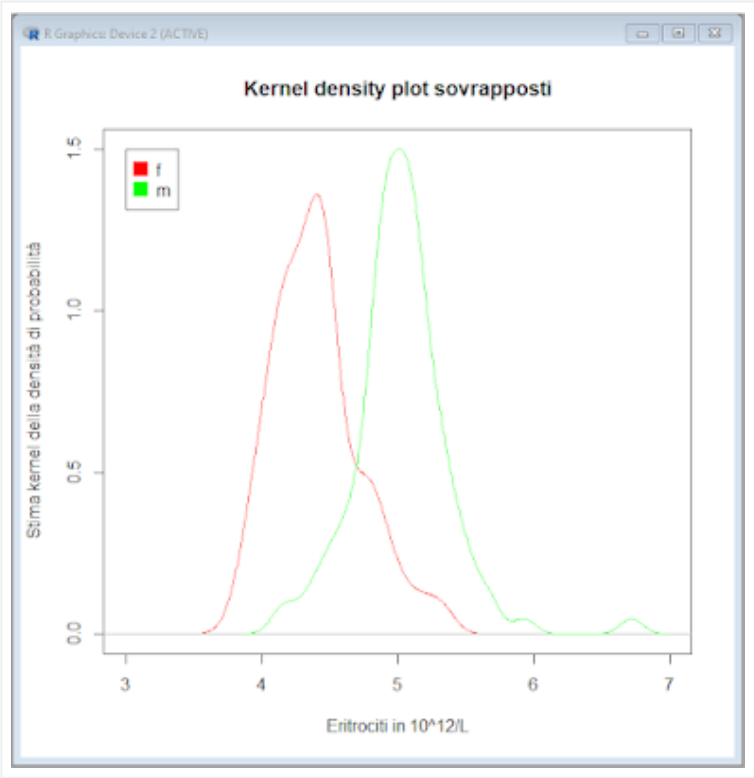
```
> density(ais$rcc)$bw
[1] 0.1425658
```

e potete notare che in effetti è superiore a **0.1** cosa che comporta un maggior smussamento del kernel density plot, che vedete più arrotondato e con una minore separazione dei picchi rispetto ai due precedenti script.

Nota bene: il kernel density plot proprio per il modo nel quale è calcolato presenta una distribuzione *bimodale* che ben si adatta a quella che si osserva nell'istogramma.

Facciamo un ulteriore passo avanti con il codice che sovrappone due kernel density plot in modo semplice e conciso. Copiate e incollate questo script nella `Console di R` e premete ↵ `Invio`:

DUE KERNEL DENSITY PLOT SOVRAPPOSTI
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
f <-as.matrix(subset(ais, sex=="f", select=c(rcc))) # sottoinsieme dei dati con sesso=f
m <-as.matrix(subset(ais, sex=="m", select=c(rcc))) # sottoinsieme dei dati con sesso=m
#
plot(density(f), xlim=c(3,7), ylim=c(0,1.5), col="red", lwd=1, main="Kernel density plot sovrapposti", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") # traccia il kernel density plot del sesso=f
#
lines(density(m), xlim=c(3,7), ylim=c(0,1.5), col="green", lwd=1) # traccia il kernel density plot del sesso=m
#
legend(3, 1.5, legend=c("f","m"), col=c("red", "green"), pt.cex=2, pch=15) # aggiunge la legenda
#



Per la funzione **subset()** dobbiamo partire dalla tabella **ais**, se nella `Console di R` digitate

str(ais)

vi comparirà la struttura della tabella

```
> str(ais)
'data.frame':   202 obs. of  13 variables:
 $ rcc   : num  3.96 4.41 4.14 4.11 4.45 4.1 4.31 4.42 4.3 4.51 ...
 $ wcc   : num  7.5 8.3 5 5.3 6.8 4.4 5.3 5.7 8.9 4.4 ...
 $ hc    : num  37.5 38.2 36.4 37.3 41.5 37.4 39.6 39.9 41.1 41.6 ...
 $ hg    : num  12.3 12.7 11.6 12.6 14 12.5 12.8 13.2 13.5 12.7 ...
 $ ferr  : num  60 68 21 69 29 42 73 44 41 44 ...
 $ bmi   : num  20.6 20.7 21.9 21.9 19 ...
 $ ssf   : num  109.1 102.8 104.6 126.4 80.3 ...
 $ pcBfat: num  19.8 21.3 19.9 23.7 17.6 ...
```

```
 $ lbm   : num  63.3 58.5 55.4 57.2 53.2 ...
 $ ht    : num  196 190 178 185 185 ...
 $ wt    : num  78.9 74.4 69.1 74.9 64.6 63.7 75.2 62.3 66.5 62.9 ...
 $ sex   : Factor w/ 2 levels "f","m": 1 1 1 1 1 1 1 1 1 1 ...
 $ sport : Factor w/ 10 levels "B_Ball","Field",..: 1 1 1 1 1 1 1 1 1 1 ...
```

che come si vede contiene 13 variabili. Le prime 11 sono variabili numeriche (`num`), le ultime due sono fattori (`Factor`), sono cioè variabili qualitative opportunamente codificate, la variabile `sex` per indicare il sesso impiegando 2 livelli di codifica (`"f"` e `"m"`), la variabile `sport` per indicare lo sport praticato, impiegando 10 livelli di codifica.

Con la funzione **subset()** sono estratti da **ais** i sottoinsiemi di dati **f** e **m** impiegando i seguenti argomenti:
→ **ais** per indicare la fonte dei dati da estrarre;
→ **sex="f"** ovvero **sex="m"** per estrarre da **ais** rispettivamente i dati corrispondenti agli atleti di sesso femminile e agli atleti di sesso machile;
→ **select=(rcc)** per estrarre da **ais** solamente i valori della variabile **rcc** che sono quelli che ci interessano per tracciare i kernel density plot.

I due kernel density plot sono tracciati con due righe di codice che impiegano:
→ la funzione **plot()** per tracciare il primo kernel density plot;
→ la funzione **lines()** per sovrapporre il secondo.

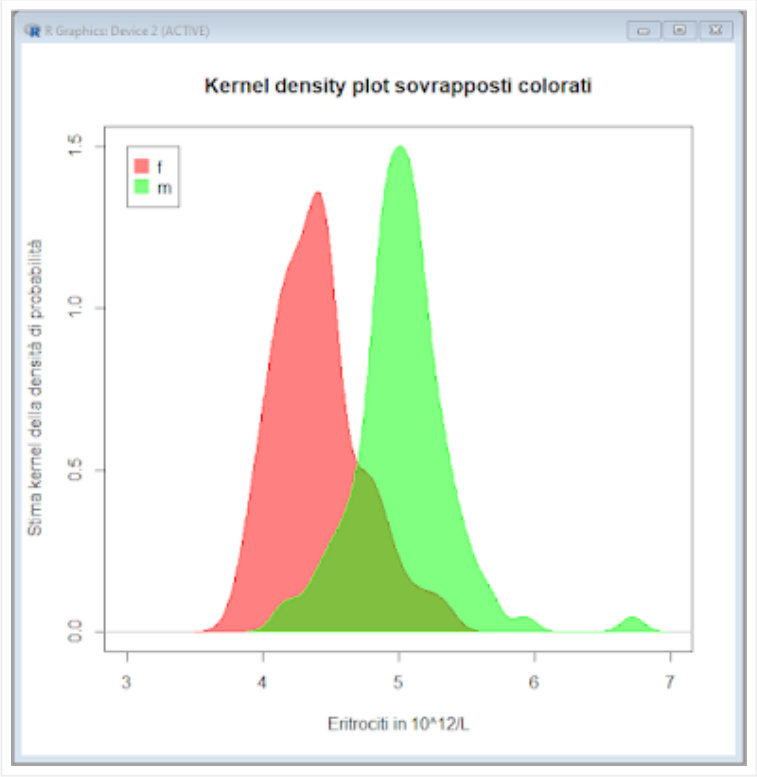
Da notare a proposito della funzione **lines()** che:
→ riporta sullo stesso grafico e unisce tra loro con segmenti di retta i punti con le coordinate `x,y` del kernel density plot calcolate dalla funzione **density()**;
→ riporta come scala degli assi gli stessi valori **xlim=c(3,7)** e **ylim=c(0,1.5)** impiegati per il primo kernel density plot;
→ traccia i segmenti di retta che descrivono il kernel density plot con il colore **col="green"** e con una linea di spessore **lwd=1**.

La funzione **legend()** [6] identifica i dati rappresentati impiegando i seguenti argomenti:
→ **3** per l'ascissa cui allineare il lato sinistro della legenda;
→ **1.5** per l'ordinata cui allineare l'angolo superiore della legenda;
→ **legend=c(...)** per l'elenco delle etichette che descrivono i dati:
→ **col=c(...)** per l'elenco dei colori corrispondenti ai dati
→ **pt.cex=2** per indicare il fattore di ingrandimento dei simboli;
→ **pch=15** per indicare il tipo di simbolo da impiegare (15=un quadrato).

Vediamo ora come sovrapporre due kernel density plot colorati e renderli visibili nella loro interezza, senza che uno nasconda parzialmente l'altro, utilizzando la trasparenza del colore. Copiate e incollate lo script nella `Console di R` e premete `↵ Invio`:

```
# DUE KERNEL DENSITY PLOT SOVRAPPOSTI COLORATI
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
f <-as.matrix(subset(ais, sex=="f", select=c(rcc))) # sottoinsieme dei dati con sesso=f
m <-as.matrix(subset(ais, sex=="m", select=c(rcc))) # sottoinsieme dei dati con sesso=m
#
plot(density(f), xlim=c(3,7), ylim=c(0,1.5), main="Kernel density plot sovrapposti
colorati", xlab="Eritrociti in 10^12/L", ylab="Stima kernel della densità di probabilità") #
traccia il kernel density plot del sesso=f
polygon(density(f), col=rgb(255,0,0,127,maxColorValue=255), border="red", lwd=1) #
colora il kernel density plot del sesso=f
#
lines(density(m), xlim=c(3,7), ylim=c(0,1.5)) # traccia il kernel density plot del sesso=m
```

polygon(density(m), col=rgb(0,255,0,127,maxColorValue=255), border="green", lwd=1) # colora il kernel density plot del sesso=m
#
legend(3, 1.5, legend=c("f","m"), col=c(rgb(255,0,0,127,maxColorValue=255), rgb(0,255,0,127,maxColorValue=255)), pt.cex=2, pch=15) # aggiunge la legenda
#



La funzione **subset()** impiegata per estrarre da **ais** i sottoinsiemi di dati **f** e **m** è stata illustrata nello script precedente.

Per sovrapporre i due kernel density plot colorati sono state impiegate le funzioni **plot()**, **density()**, **polygon()** e **lines()** anch'esse già illustrate in precedenza.

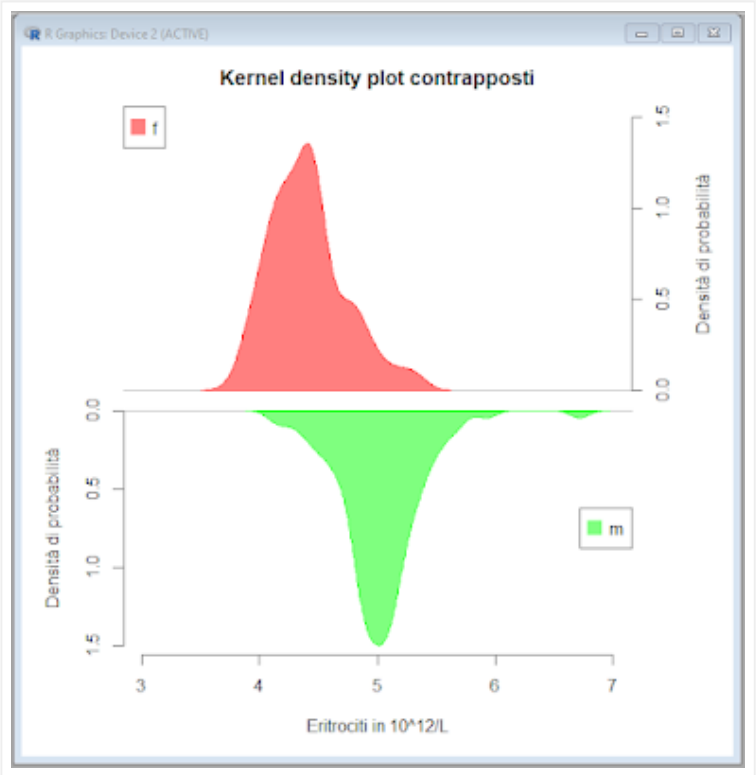
Nelle funzioni **poligon()** e **legend()** il colore è stato specificato con la funzione **rgb(n,n,n,n)** nella quale:
→ il primo **n**umero indica il rosso (**r**ed);
→ il secondo il verde (**g**reen);
→ il terzo il blu (**b**lu);
→ il quarto il grado di trasparenza del colore impiegato.

Ponendo **maxColorValue=255** questi quattro valori possono essere espressi in una scala compresa tra **0** e **255**. Così ad esempio i valori di **rgb** uguali a **255,0,0,127** del primo dei due kernel density plot stanno per rosso al 100%, no verde, no blu, trasparenza del colore uguale al 50%. Digitale **help(rgb)** per la documentazione completa della funzione.

Si possono anche realizzare due kernel density plot contrapposti. Copiate e incollate lo script nella `Console di R` e premete `↵ Invio`:

DUE KERNEL DENSITY PLOT CONTRAPPOSTI
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
windows() # apre e inizializza una nuova finestra grafica
#
f <-as.matrix(subset(ais, sex=="f", select=c(rcc))) # estrae il sottoinsieme di ais con sesso=f
m <-as.matrix(subset(ais, sex=="m", select=c(rcc))) # estrae il sottoinsieme di ais con sesso=m

par(mfrow=c(2,1)) # grafici organizzati in due righe e una colonna
#
par(mar=c(0,5,3,5)) # adatta i margini al grafico superiore
plot(density(f), frame.plot=FALSE, xaxt="n", yaxt="n", xlim=c(3,7), ylim=c(0,1.5), main="Kernel density plot contrapposti", xlab="", ylab="") # traccia il kernel density plot del sesso=f
axis(4) # riporta l'asse delle y del kernel density plot sulla destra
mtext("Densità di probabilità", side=4, line=3) # aggiunge la legenda all'asse delle y sulla destra
polygon(density(f), col=rgb(1,0,0,0.5), border="red", lwd=1) # colora il kernel density plot
legend("topleft", legend=c("f"), col=c(rgb(1,0,0,0.5)), pt.cex=2, pch=15) # riporta la legenda
#
par(mar=c(5,5,0,5)) # adatta i margini al grafico inferiore
plot(density(m), frame.plot=FALSE, xlim=c(3,7), ylim=c(1.5,0), main="", xlab="Eritrociti in 10^12/L", ylab="Densità di probabilità") # traccia il kernel density plot del sesso=m
polygon(density(m), col=rgb(0,1,0,0.5), border="green", lwd=1) # colora il kernel density plot
legend("right", legend=c("m"), col=c(rgb(0,1,0,0.5)), pt.cex=2, pch=15) # riporta la legenda
#



Si tratta di una estetica alternativa a quella dello script precedente, che può piacere o non piacere, e che viene realizzata in modo molto semplice con le funzioni:
→ **plot()** per tracciare il grafico;
→ **polygon()** per colorare il grafico;
→ **legend()** per aggiungere la legenda.

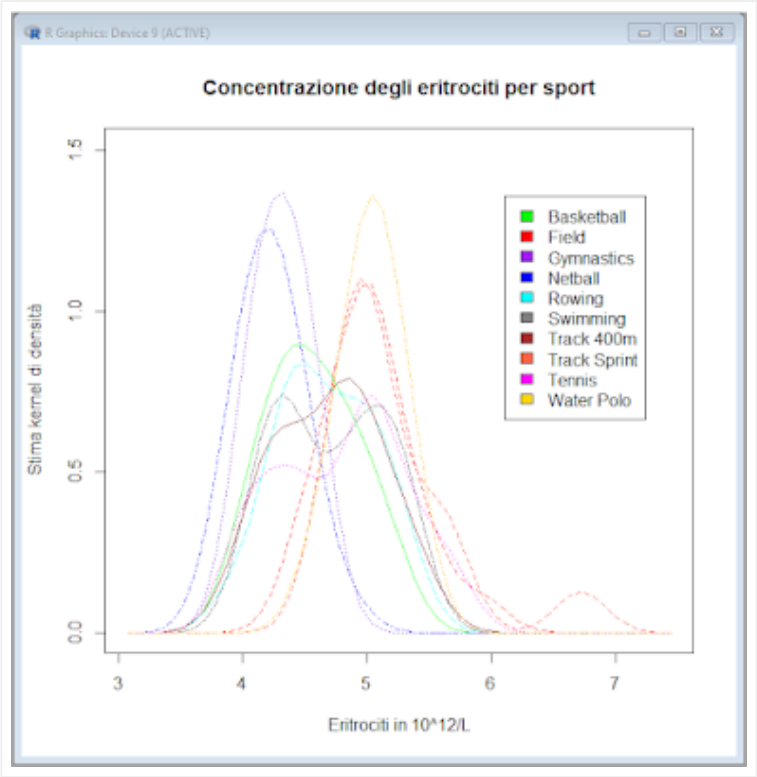
Per realizzare il grafico queste tre funzioni sono integrate con i seguenti accorgimenti:
→ **par(mfrow=c(2,1))** per disporre i grafici in due righe per una colonna quindi in definitiva uno sopra l'altro;
→ **par(mar=c(0,5,3,5))** per eliminare il margine inferiore e aumentare il margine destro al grafico superiore;
→ **xaxt="n"** per togliere l'asse delle x al grafico superiore (è sufficiente l'asse delle ascisse riportato al grafico inferiore);

→ **yaxt="n"** per togliere al grafico superiore l'asse delle `y` che per default verrebbe riportato sulla sinistra;
→ **axis(4)** per riportare l'asse delle `y` del grafico superiore sulla destra;
→ **mtext()** per aggiungere la legenda **"Densità di probabilità"** all'asse delle `y` di destra (**side=4**) sulla riga 3 (**line=3**);
→ **par(mar=c(5,5,0,4))** per eliminare il margine superiore e aumentare il margine destro al grafico inferiore;
→ **ylim=c(1.5,0)** necessario per invertire nel grafico inferiore l'asse delle y rispetto al grafico superiore, nel quale abbiamo **ylim=c(0,1,5)**.

Da notare che nelle funzioni **polygon()** e **legend()** per definire il colore viene impiegata la funzione **rgb(n,n,n,n)** ma, contrariamente a quanto avevamo fatto nello script precedente, questa volta non viene specificato l'argomento `maxColorValue` che pertanto qui assume il valore di default, che è uguale a `1`. Quindi i valori **n** assegnati nell'ordine al rosso (**r**ed), al verde (**g**reen), al blu (**b**lu) e al grado di trasparenza del colore, per definizione possono variare solamente tra `0` e `1`. In questa scala il valore **0.5** del grado di trasparenza equivale al valore **127** che nello script precedente era espresso in una scala da **0** a **255**.

Qualora si debbano sovrapporre molti kernel density plot, può risultare comodo impiegare le funzioni contenute nel pacchetto **sm**. Accertatevi di avere installato il pacchetto altrimenti dovete scaricarlo e installarlo dal **CRAN**. Ora copiate e incollate questo script nella `Console di R` e premete `↵ Invio`:

```
# KERNEL DENSITY PLOT MULTIPLI SOVRAPPOSTI con il pacchetto sm
#
library(DAAG) # carica il pacchetto DAAG che include il set di dati ais
library(sm) # carica il pacchetto
windows() # apre e inizializza una nuova finestra grafica
#
sport <- c("B_Ball", "Field", "Gym", "Netball", "Row", "Swim", "T_400m", "T_Sprnt",
"Tennis", "W_Polo") # vettore che riporta la codifica impiegata per la variabile/fattore sport
etichette <- c("Basketball", "Field", "Gymnastics", "Netball", "Rowing", "Swimming",
"Track 400m", "Track Sprint", "Tennis", "Water Polo") # vettore con le etichette da assegnare
agli sport
colori <- c("green", "red", "purple", "blue", "cyan", "grey50", "brown", "tomato",
"magenta", "gold") # vettore con i colori da assegnare agli sport
#
sm.density.compare(ais$rcc, ais$sport, col=colori, xlab="Eritrociti in 10^12/L",
ylab="Stima kernel di densità") # traccia i kernel density plot separati per sport
#
title(main="Concentrazione degli eritrociti per sport praticato") # aggiunge il titolo
legend(locator(1), levels(factor(ais$sport, levels=sport, labels=etichette)), fill=colori) #
predispone la legenda con etichette e colori
#
# resta in attesa del click con il tasto sinistro del mouse nel punto in cui si desidera posizionare la
legenda
#
```



Il punto di forza è che se i dati sono associati a una variabile qualitativa per la quale sono previsti `N` livelli di codifica, vengono generati automaticamente sia gli `N` kernel density plot corrispondenti sia la legenda necessaria per identificarli. Inoltre la legenda può essere posizionata nel grafico in un punto a piacere con un semplice `click` del mouse.

Nel nostro caso per rappresentare la concentrazione degli eritrociti rilevata nei 10 sport praticati dagli atleti australiani in altrettanti kernel density plot procediamo come segue:
→ nel vettore **sport=c(...)** riportiamo la codifica dei livelli impiegata nella variabile (fattore) `sport`;
→ nel vettore **etichette=c(...)** riportiamo nello stesso ordine le scritte che vogliamo impiegare per decrivere ciascuno sport;
→ nel vettore **colori=c(...)** riportiamo nello stesso ordine i colori che vogliamo applicare a ciacuno sport nel kernel density plot e nella legenda corrispondente;
→ con la funzione **sm.density.compare()** tracciamo i kernel density plot per la variabile **ais$rcc** suddividendoli per sport (**ais$sport**) impiegando i colori definiti nel vettore **colori**;
→ con la funzione **title()** riportiamo il titolo del grafico;
→ con la funzione **legend()** riportiamo riportiamo nella legenda, per la variabile (fattore) **ais$sport**, i livelli definiti nel vettore **sport**, con le etichette definite nel vettore **etichette** e i colori definiti nel vettore **colori**.

Una volta che sono comparsi i grafici, posizionate il puntatore del mouse dove volete che compaia la legenda e fate `click` con il tasto sinistro del mouse per farla comparire, terminando così lo script. Da notare che il codice riportato non consente di riposizionare dinamicamente la legenda: se non si è soddisfatti della sua posizione, è necessario rieseguire l'intero script e fare nuovamente `click` con il tasto sinistro del mouse nel nuovo punto in cui la si vuole posizionare.

Come al solito gli script riportati possono essere immediatamente riutilizzati sostituendo **ais$rcc** con il nome della propria variabile e con il semplice adattamento di titoli ed etichette degli assi [7].

---

[1] I metodi parametrici sono basati sull'assunto che i dati abbiano una distribuzione gaussiana e utilizzano i "parametri" *media* (misura di posizione) e *deviazione standard* (misura di dispersione) nella sintesi dei dati e nell'esecuzione dei test statistici. I metodi non parametrici invece non prevedono assunti riguardo la distribuzione dei dati e per definizione non impiegano i citati "parametri" nella sintesi dei dati e nell'esecuzione dei test statistici.

[2] Vedere il post Scale nominali, scale ordinali e scale numeriche.

[3] Vedere il post Il set di dati ais. La concentrazione degli eritrociti viene espressa in 10^{12}/L o in 10^{6}/μL, le due unità di misura impiegate nella pratica, che sono numericamente identiche. Nel post trovate anche come caricare i dati della tabella senza impiegare il pacchetto **DAAG**.

[4] Qui vediamo i primi quattro fattori di forma, per i fattori **"biweight"**, **"cosine"** e **"optcosine"**, che forniscono risultati molto simili tra loro e praticamente identici a **"gaussian"**, si rimanda alla documentazione di **R**.

[5] La distribuzione *unimodale* è per definizione quella della distribuzione gaussiana, la distribuzione *bimodale* tipicamente si osserva quando una variabile/serie di misure è composta da due sottoinsiemi aventi *mode* (ma anche medie) differenti, mentre una distribuzione *plurimodale* (o *polimodale*) è quella con mode multiple.

[6] Trovate altri esempi di legende nel post Aggiungere la legenda a un grafico.

[7] Un poco di lavoro in più è necessario per adattare lo script per la generazione di kernel density plot multipli sovrapposti con il pacchetto **sm**, nel quale **ais$rcc** deve essere sostituito con il nome della `tabella$variabile` da analizzare, **ais$sport** deve essere sostituito con il nome della `tabella$variabile` che contiene i fattori dai quali generare i kernel density plot, e infine devono essere adattati opportunamente il vettore **sport** che contiene i nomi dei fattori e i vettori **etichette** e **colori**.